Mondragon Unibertsitatea

Faculty of Engineering

Datuen Ingeniaritza

- Data Engineering -

- Ingeniería de Datos -

1

Kontenedoreak

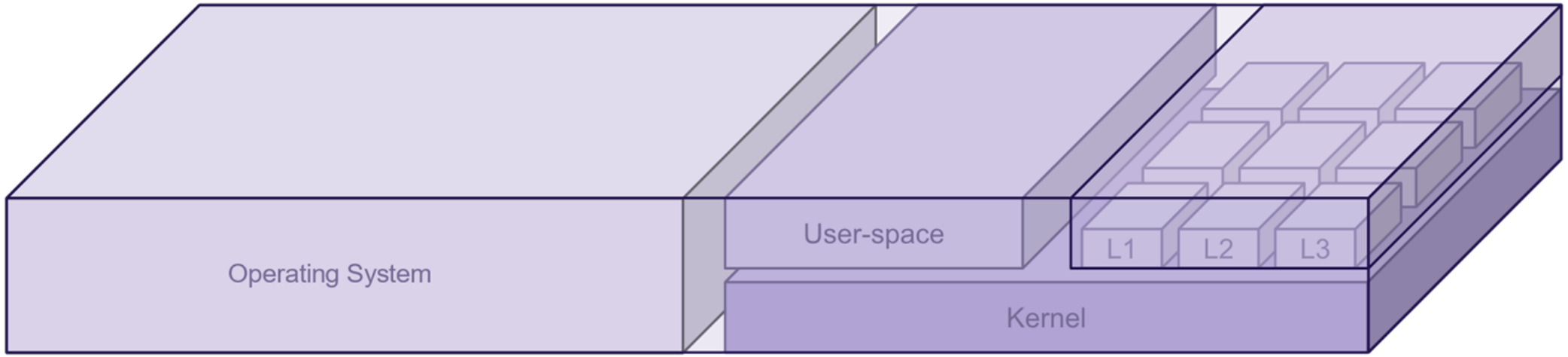
Kontenedoreak

Zer dira kontenedoreak?

Zeintzuk dira kontenedoreen osagaiak?

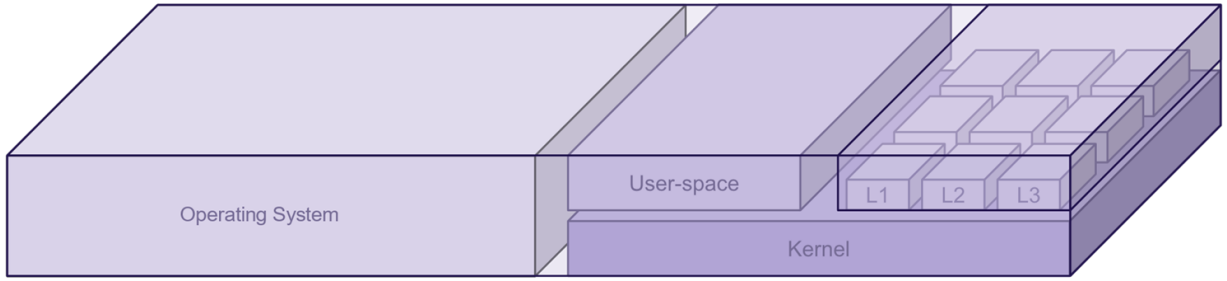
Kontenedoreak – Sistema Eragilea

- Sistema eragilea
  - Bi osagai edo geruza nagusi dituzte
    - Kernela
    - Erabiltzaile espazioa/geruza librerias que ofrece el OS principalmente. nos dejan interactuar con el kernel usando syscall



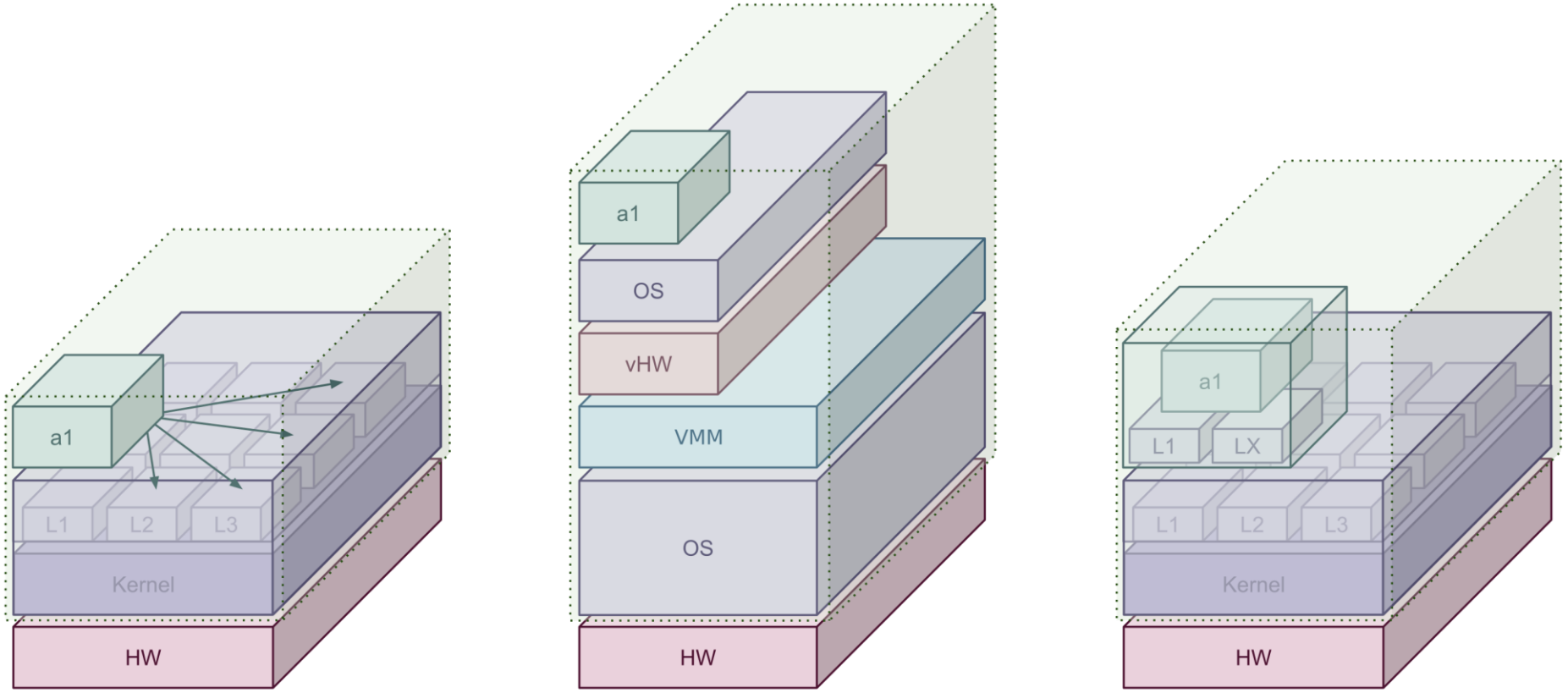
Kontenedoreak – Sistema Eragilea (geruzak)

- Geruza bakoitzaren ardurak/eginbeharrak
  - Kernela
    - Prozesuen kontrol eta kudeaketa
      - Hariak, *threads*
      - Konkurrentzia
      - Prozesuen arteko komunikazioa, *Inter Process Communication*
    - Memoriaren kudeaketa
      - Zenbakitzea, *paging*
      - Segmentazioa
      - Memoria birtuala
    - I/O (irteera/sarrera) kudeaketa
      - Interrupzioak
      - Almazenamendua
        - Fitxategi kudeaketa
      - Erlojua
      - Terminala
      - Interfaze grafikoa
      - Sarea
      - Energia
  - Erabiltzaile espazioa/geruza
    - Beste osagai batzuen artean
      - Libreria estatikoak
      - Libreria dinamikoak
      - Aplikazioak



Kontenedoreak – Alderaketa

- Zein ezberdintasun dago aplikazio bat zuzenean sistema eragilearen gainean, makina birtual baten gainean edo kontenedore batean *instalatzean*?



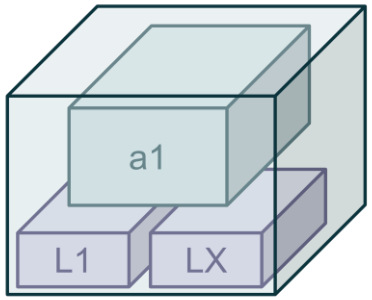
Kontenedoreak – Oinarriak

- Zerk osatzen du kontenedore irudi bat?
  - Interesatzen zaizkigun aplikazioek
  - Aplikazio hauek behar dituzten libreriak

- Martxan dagoenean, zer da kontenedore instantzia bat?

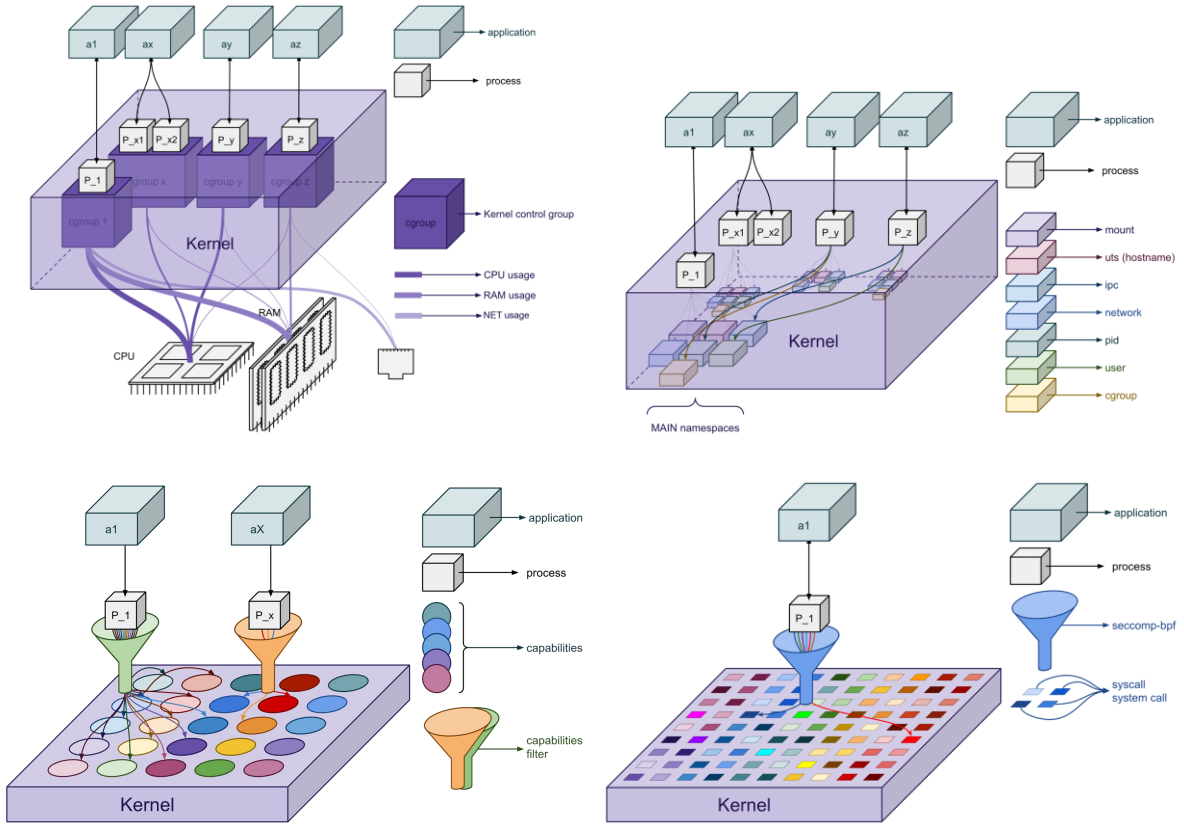
de la imagen una instanci, como en java

  - Prozesu multzo bat
    - Sistemako gainontzeko prozesuetatik isolatua ondorengo planoetan
      - Erabiltzaileak
      - Sarea
      - Fitxategi sistema
      - Prozesu identifikadoreak
      - CPU eta memoria limiteak



Kontenedoreak – Oinarriak

- Zein kernel ezaugarrik ahalbidetzen dituzte kontenedoreak?
  - cgroups, *control groups*
  - namespaces
  - capabilities
  - syscalls / seccomp-bpf
  - chroot
  - copy-on-write fitxategi sistemak



Kontenedoreak – Exekuzio motoreak

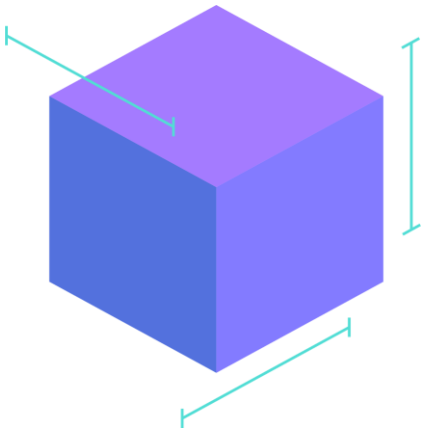
Mondragon Unibertsitatea
Faculty of Engineering

	Docker	podman	containerd	CRI-O	LXC/LXD	Windows Containers	OpenVZ
Exekuzio ingurunea	Zerbitzua	Ez du zerbitzurik behar	Docker eta Kubernetes-en exekuzio motorra	Kubernetes-en exekuzio motorra	Linux-en kontenedore kudeatzailea	Berezko Windows euskarria	Linux-en oinarritutako birtualizazioa
Ezaugarri nagusiak	Ekosistema zabala, erabilerraza, Docker Hub	Docker-en CLI-arekin bateragarria	Sinplea, sendoa, CNFC-ko parte	Kubernetes-en CRI inplementatzen du	Sistema kontenedoreak	Windows Server-ekin bateratzen da	Errendimendu altua
Erabilera nagusia	Garapena, testatzea, produkzioa	Garapena, testatzea, exekuzio lokala	Docker eta Kubernetes-en exekuzio motorra	Kubernetes inguruneak	Garapena, testatzea, aplikazio zaharren kontenedoreak	Windows-en oinarritutako aplikazioak, enpresa inguruneak	*Hosting*-ak, VM bezala kontenedore erabilera
Kodigo askea	Bai (alderdi komertzialekin)	Bai	Bai	Bai	Bai	Ez	Bai
OCI bateragarria	Bai	Bai	Bai	Bai	Ez	Ez	Ez

Kontenedoreak – OCI

- Zer da OCI?
  - OCI edo *Open Container Initiative*, 2015ean sortu zen ondorengo atalen inguruan estandar eta espezifikazioak sortzeko
    de que formato tiene que ser la imagen
    - kontenedore irudiak, *image-spec*
    - *runtime*, *runtime-spec*
      - Dagoeneko zabalduta dugun fitxategi sistema nola exekutatu

- OCIri esker podman eta docker bateragarriak dira

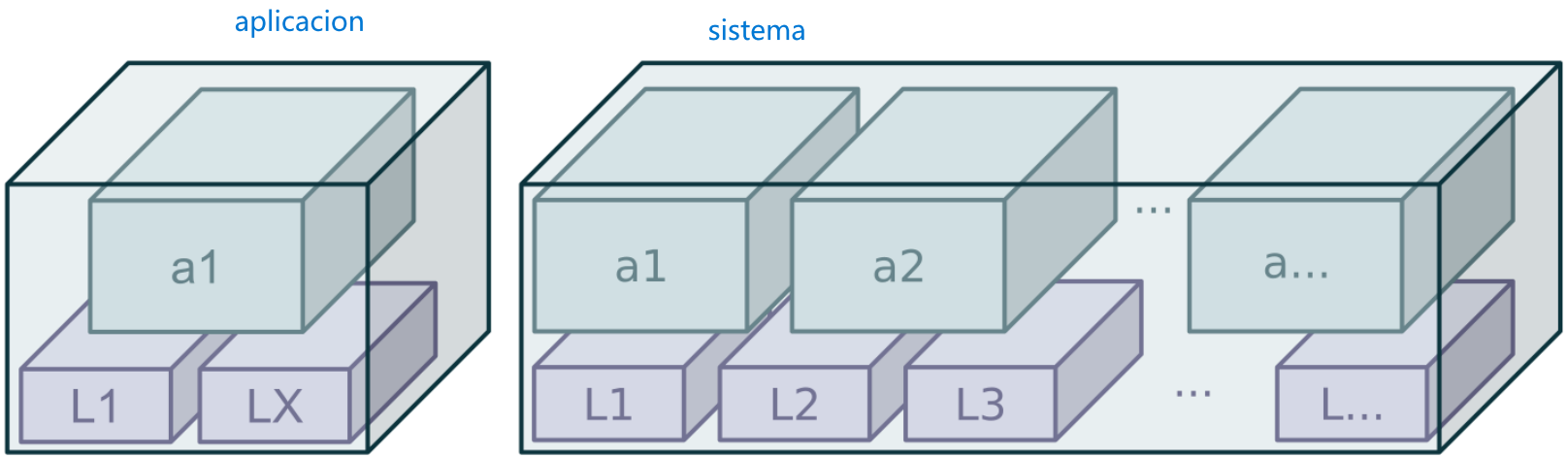


https://opencontainers.org/

Kontenedoreak – Aplikazio/Sistema kontenedoreak

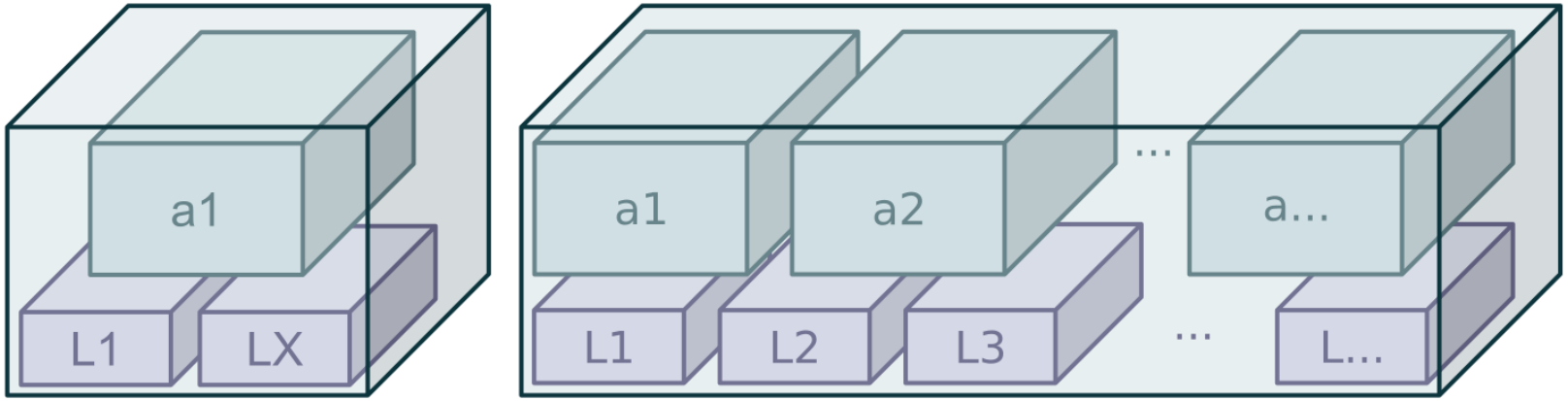
- Zenbat mota ezberdinetako kontenedoreak daude?
  - Nagusiki, bi mota ezberdindu ditzakegu
    - Sistema kontenedoreak
    - Aplikazio kontenedoreak

usan las mismas caracteristcas del kernel, lo diferente es el objetivo (y el tamaño de las imagenes)
- sistema: todo el sistema menos el kernel en el contenedor (por eso dentro de proxmox podemos hacer un contenedor de debian)
- aplicacion: una unica aplicacion en el contenedor



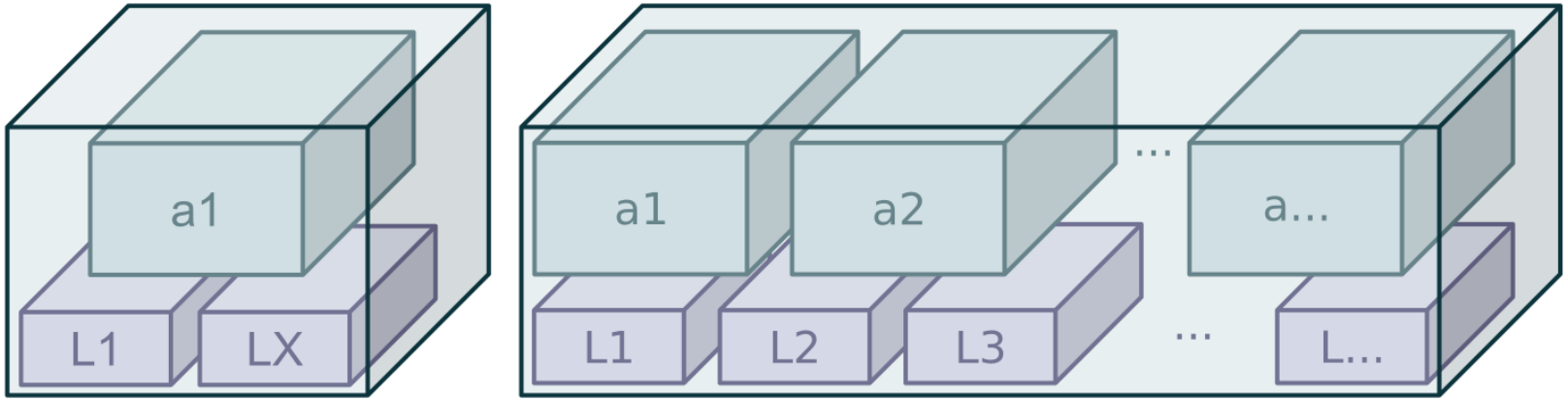
Kontenedoreak – Aplikazio/Sistema kont. alderaketa

- Zein da aplikazio kontenedore eta sistema kontenedore baten arteko ezberdintasuna?
  - Aplikazio kontenedorearen helburua aplikazio bakarra edo beharrezkoak bakarrik diren aplikazioak eta hauen dependentziak exekutatzea da
  - Sistema kontenedorearen helburua aldiz, kernela albo batera utzita, sistema eragile baten erabiltzaile espazioko aplikazio eta dependentzia guztiak exekutatzea da



Kontenedoreak – Aplikazio/Sistema kont. alderaketa

- Zer dute aplikazio kontenedore eta sistema kontenedoreek amankomunean?
  - Bi motatako kontenedoreak kernelaren ezaugarri berdinetan oinarritzen dira
    - cgroups, control groups
    - namespaces
    - capabilities
    - syscalls / seccomp-bpf
    - chroot



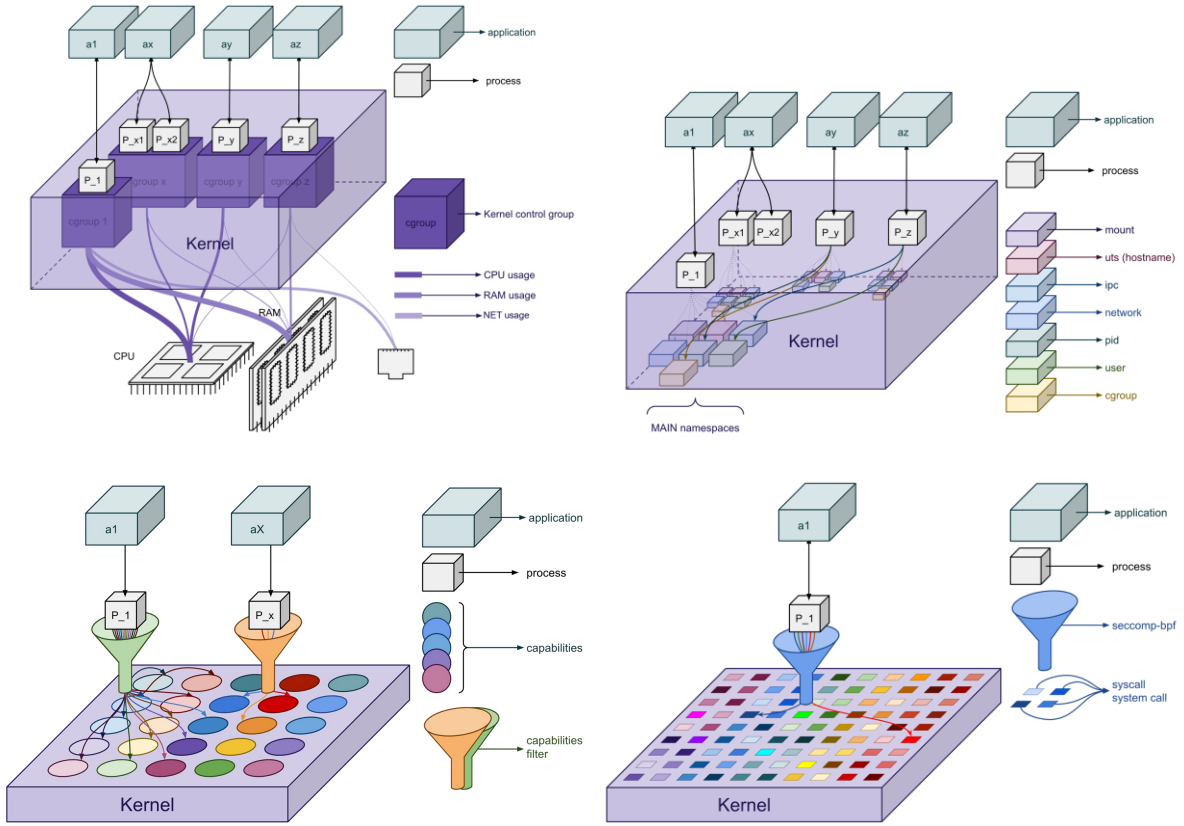
1.1

Kernel ezaugarriak

Kontenedoreak

Kernel ezaugarriak

- Ondorengo kernel ezaugarriek ahalbidetzen dituzte kontenedoreak
  - cgroups, *control groups*
  - namespaces
  - capabilities
  - syscalls / seccomp-bpf
  - chroot
  - copy-on-write fitxategi sistemak

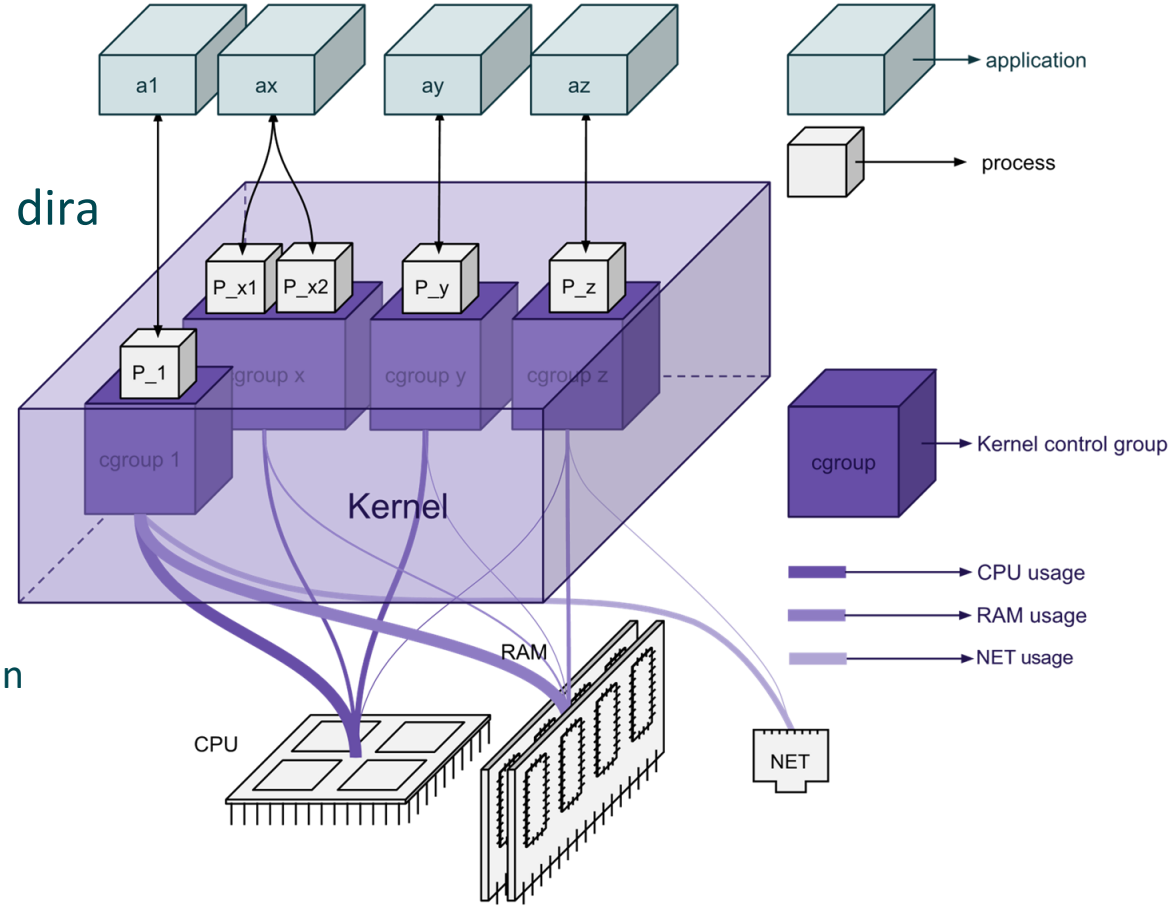


Kernel ezaugarriak – cgroups

karpeta batean dauden prozesu multzoak limitatu?

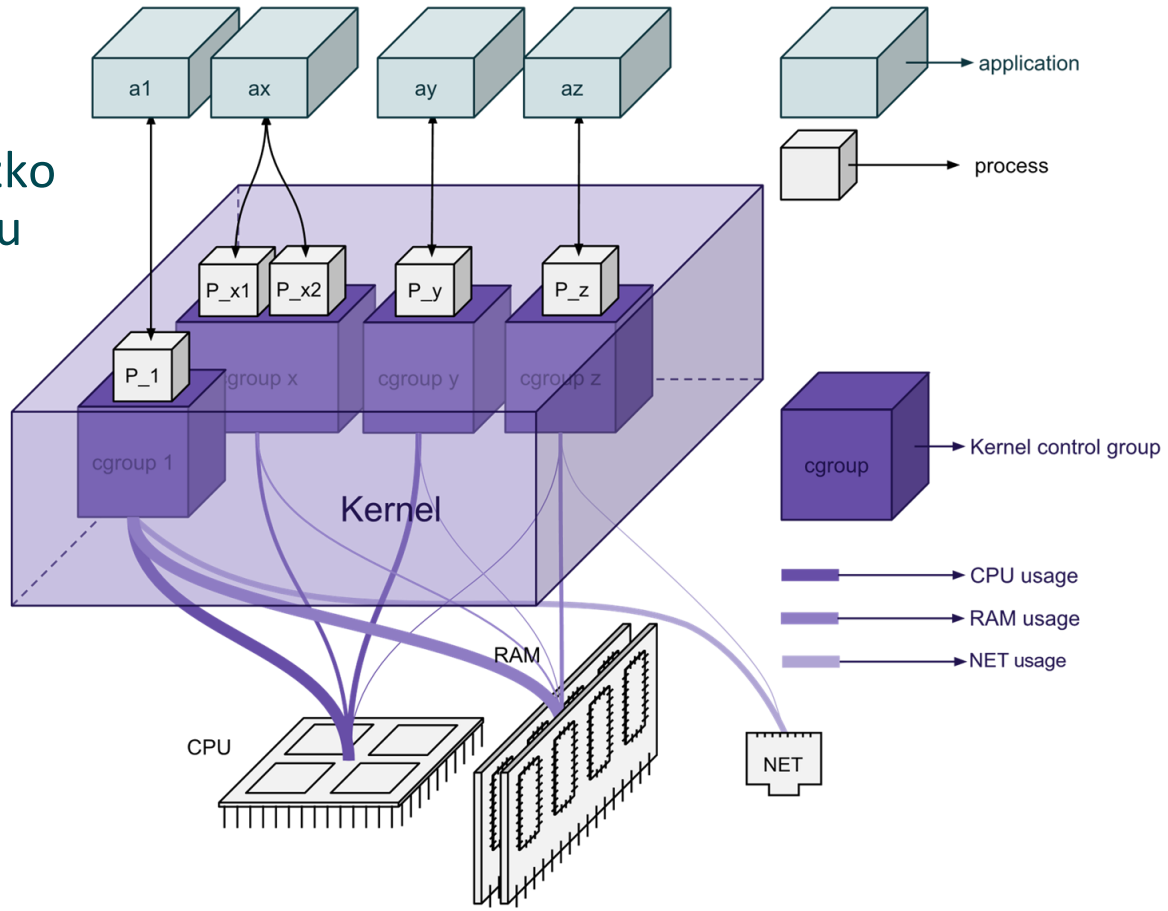
si un proceso dentro del control group tiene un hijo, el hijo estara dentro del control group
dentro de un cg puede haber otro cg
todos los procesos de un sistema estan en un cg, siempre se empieza con uno

- *Control groups*
  - Prozesu talde edo multzoak
    - Kudeatu
    - Mugatu
    - Ikuskatu
  - Ondorengoak mugatzeko erabiltzen dira
    - CPU
    - Memoria
    - Sareko I/O
    - I/O
    - PID kopurua
  - Bi bertsio existitzen diren arren
    - Indarrean bigarren bertsioa dago
      - Nahiz eta ez dituen lehenengo bertsioaren kontrol guztiak jasaten



Kernel ezaugarriak – cgroups

- *Control groups*
  - Sistema hierarkikoa da
  - Prozesu jakin bat aldiro *cgroup* bakarrean egon daiteke
  - Prozesu bat sortzen denean, jatorrizko prozesuaren cgroup-a heredatzen du



Kernel ezaugarriak – cgroups

Mondragon Unibertsitatea
Faculty of Engineering

Adibidea 1

Kernel ezaugarriak – cgroups

- Sistemako cgroup-ak ikuskatu
  - Sistemak egiten duen cgroup-en erabilera ikusiko dugu
    - cgroup-ak /sys/fs/cgroup barruan definitzen dira

```
$/# ls -l /sys/fs/cgroup/
```

    - systemd init sistemak hainbat cgroup sortzen ditu modu hierarkikoan

```
# systemctl status
```

    - cgroup bakoitzean dauden prozesu ID-ak inprimitu

```
# find /sys/fs/cgroup/ -name cgroup.procs | xargs -I % sh -c 'echo %; cat %'
```

Kernel ezaugarriak – cgroups

- Estres testa, *stress test*
  - CPUaren erabilera 100%-ean jartzean datza, konputazio gailua jokoz kanpo utziz
  - Lehenik eta behin cgroup berri bat sortuko dugu, ‘st’ deituko diogu

```
1 | # mkdir /sys/fs/cgroup/st
```

    - Zer dago sortu berri dugun karpetaren barruan?

```
1 | # ls -l /sys/fs/cgroup/st/
```

  - Ondoren, eta beste terminal batetik, bash prozesu berri bat sortuko dugu

```
2 | $ bash
```

    - Zein da bash prozesu berri honen PID-a?

```
2 | $ echo $$ edo ps eta bigarren bash prozesuaren PID-a
```

Kernel ezaugarriak – cgroups

- Estres testa, *stress test*
  - CPUaren erabilera 100%-ean jartzean datza, konputazio gailua jokoz kanpo utziz
  - Bash prozesua sortu dugun cgroup-ean sartuko dugu

```
# echo <bash_prozesuaren_PIDa> >> /sys/fs/cgroup/st/cgroup.procs
```

  - ‘st’ cgroup-ari CPU erabilera maximoa %50-ean mugatzeko esango diogu

```
# echo 50000 100000 /sys/fs/cgroup/st/cpu.max
```

  - Estres testa burutuko dugu CPU erabilera ikuskatzen dugun bitartean

```
# top
```

```
$ dd if=/dev/zero of=/dev/null
```

    - Zein da dd prozesuaren CPU erabilera?

Kernel ezaugarriak – cgroups

- Estres testa, *stress test*
  - CPUaren erabilera 100%-ean jartzean datza, konputazio gailua jokoz kanpo utziz
  - Nola mugatu dezakegu %10-era CPU-aren erabilera maximoa?

```
# echo 10000 100000 /sys/fs/cgroup/st/cpu.max
```

    - Zein da dd prozesuaren CPU erabilera?

Kernel ezaugarriak – cgroups

- Adarkatze bonba, *fork bomb*
  - Sisteman ahalik eta prozezu gehien sortzean datza, konputazio gailua jokoz kanpo utziz
  - Lehenik eta behin cgroup berri bat sortuko dugu, ‘fb’ deituko diogu

```
# mkdir /sys/fs/cgroup/fb
```

  - Ondoren, eta beste terminalean dugun bash prozesua cgroup-ean sartuko dugu

```
# echo <bash_prozesuaren_PIDa> >> /sys/fs/cgroup/fb/cgroup.procs
```

    - Aurreko cgroup-ean ere jarraitzen al du?

```
# cat /sys/fs/cgroup/st/cgroup.procs
```

Kernel ezaugarriak – cgroups

- Adarkatze bonba, *fork bomb*
  - Sisteman ahalik eta prozezu gehien sortzean datza, konputazio gailua jokoz kanpo utziz
  - cgroup-ari izan ditzakeen PID maximoa 10 izan dadila esango diogu

```
# echo 10 > /sys/fs/cgroup/fb/pids.max
```

  - Adarkatze bonba jaurtiko dugu PID kopurua ikuskatzen dugun bitartean

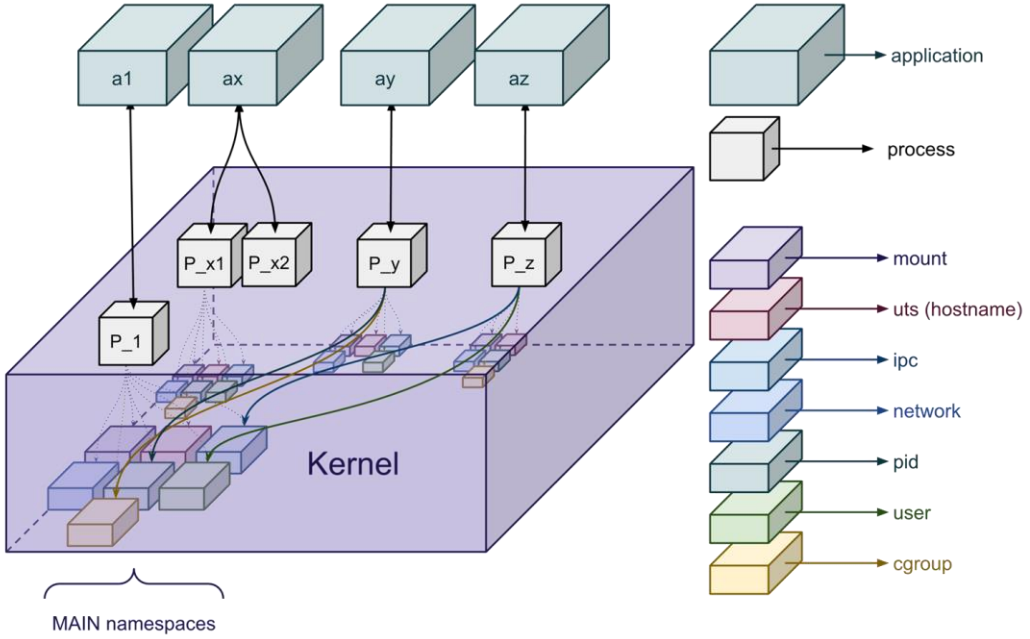
```
# watch -n 1 'ps a'
```

```
$ :(){ :|:& };:
```

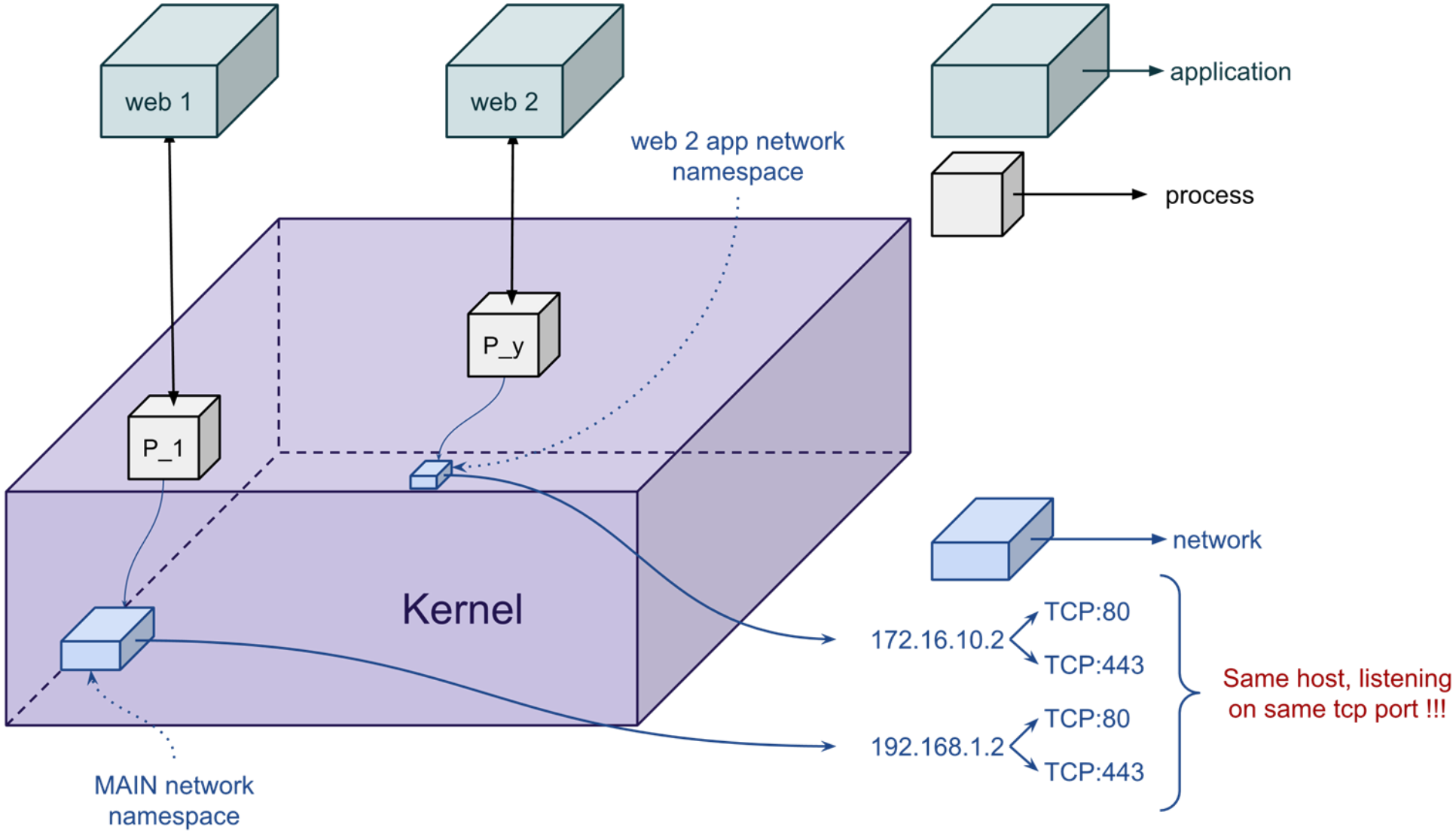
    - Zenbat bash prozesu sortu dira ‘fb’ cgroup-ean?

Kernel ezaugarriak – namespaces

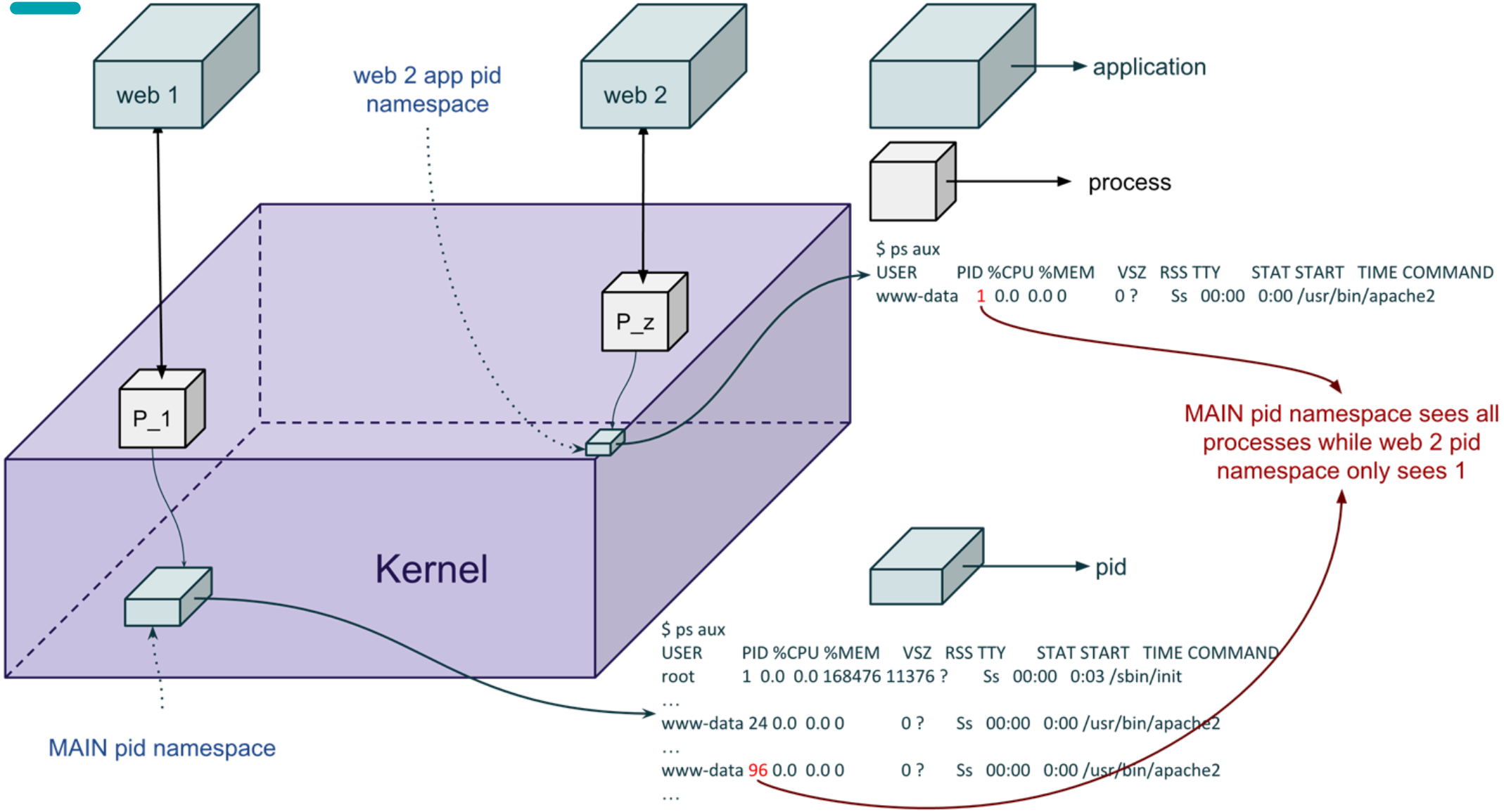
- *Namespaces* sistema errekurtsoak aislatzeko
  - Isolamentu bidezko kerneleko errekurtsoen zatikatzea
  - Egun, zortzi existitzen dira
    1. *mount*
       - Muntatuta dauden eta ikus daitezkeen fitxategi sistemak
    2. *uts*
       - *hostname*-a eta domeinu izen isolamendua
    3. *ipc*
       - Prozesuen arteko komunikazio isolamendua
    4. *network*
       - IP helbide, *route* taula, *socket*, konexioen jarraipen, suebaki isolamendua
    5. *pid*
       - Prozeduen ID-en isolamendua
    6. *user*
       - Erabiltzaile eta taldeen ID isolamendua
    7. *cgroup*
       - Ikusi eta erabili daitezkeen *cgroup*-en isolamendua
    8. *time*
       - Ikusi daitezkeen sistemako orduen eta denboren (sistema hasi deneko denbora…) isolamendua



Kernel ezaugarriak – namespaces



Kernel ezaugarriak – namespaces



Kernel ezaugarriak – namespaces

Mondragon Unibertsitatea
Faculty of Engineering

Adibidea 2

Kernel ezaugarriak – namespaces

Mondragon Unibertsitatea
Faculty of Engineering

- Sistemako namespace-ak ikuskatu
  - Sistemak egiten duen namespace-en erabilera ikusiko dugu

```
$/# lsns
```

  - *uts* namespacea erabiliko dugu prozesu bati *hostname*-a ezberdina dela ikustarazteko
    - Zein da sistemaren *hostname*-a?

```
# hostname
```

  - *uts* namespace berri baten dagoen bash prozesu berri bat sortuko dugu

```
# unshare --uts bash
```

  - Prozesu berri honi *hostname*-a aldatuko diogu

```
# hostname izenberria
```

    - *hostname*-a aldatu da?

Kernel ezaugarriak – namespaces

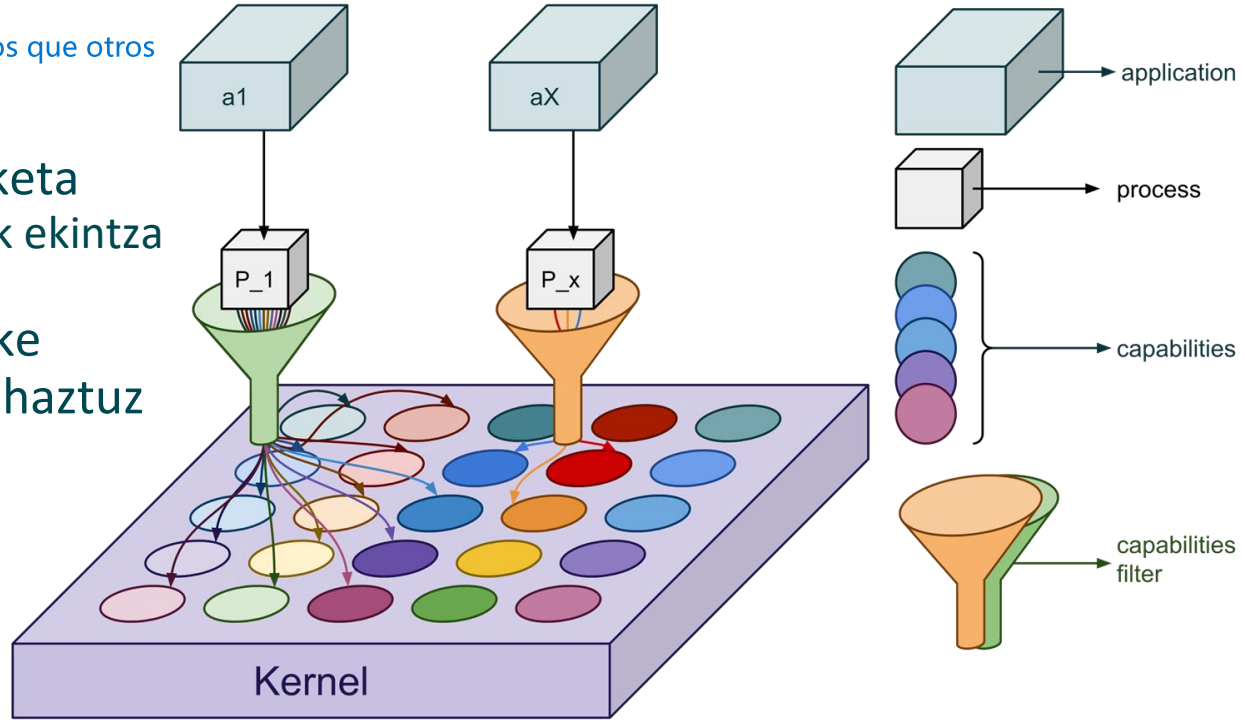
```
# bash
```

- Eta orain?

Kernel ezaugarriak – capabilities

rooten pribiligio guztiak zati txikietan zatitu. algunos capabilities tienen mas permisos que otros

- *capabilities*
  - root erabiltzailearen pribilegioen zatiketa
    - Horrela root izan gabe, prozesu batek ekintza pribilegiatuak burutu ditzake
  - Exekuzio hariei, *thread*, esleitu daitezke hari horrek egin ditzakeen ekintzak zehaztuz
  - Fitxategiek *capability*-ak ezarrita izan ditzakete
    - Adibidez /bin/ping
      - getcap /bin/ping



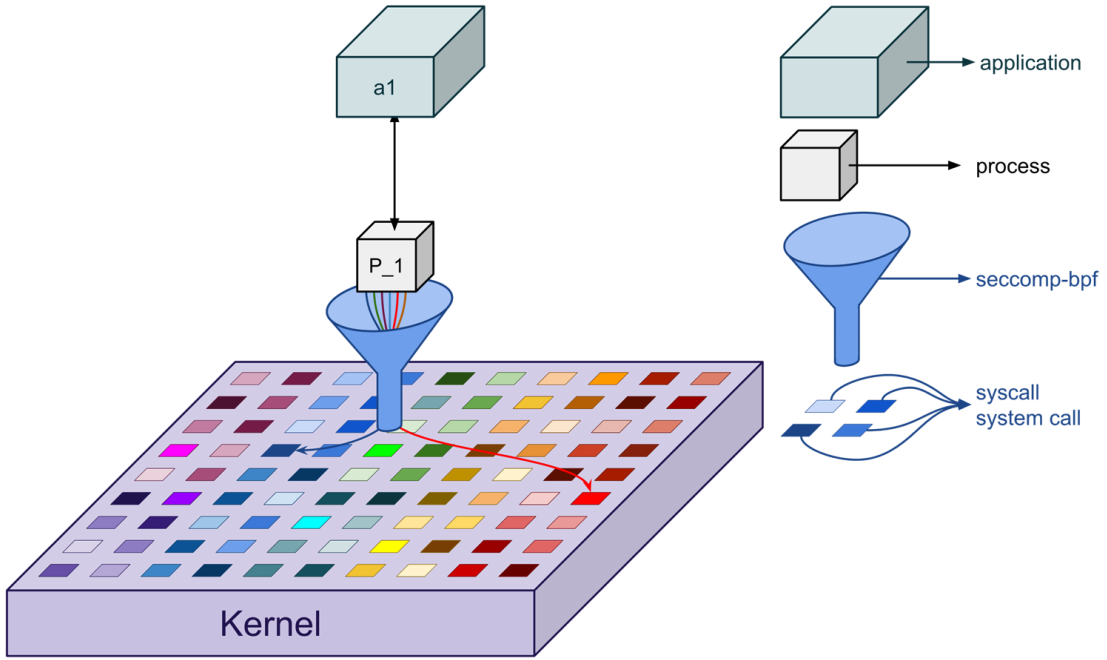
CAP_CHOWN
CAP_DAC_OVERRIDE
CAP_DAC_READ_SEARCH
CAP_FOWNER
CAP_FSETID
CAP_FS_MASK
CAP_KILL
CAP_SETGID
CAP_SETUID
CAP_SETPCAP

CAP_LINUX_IMMUTABLE
CAP_NET_BIND_SERVICE
CAP_NET_BROADCAST
CAP_NET_ADMIN
CAP_NET_RAW
CAP_IPC_LOCK
CAP_IPC_OWNER
CAP_SYS_MODULE
CAP_SYS_RAWIO
CAP_SYS_CHROOT

CAP_SYS_PTRACE
CAP_SYS_PACCT
CAP_SYS_ADMIN
CAP_SYS_BOOT
CAP_SYS_NICE
CAP_SYS_RESOURCE
CAP_SYS_TIME
CAP_SYS_TTY_CONFIG
CAP_MKNOD
CAP_LEASE

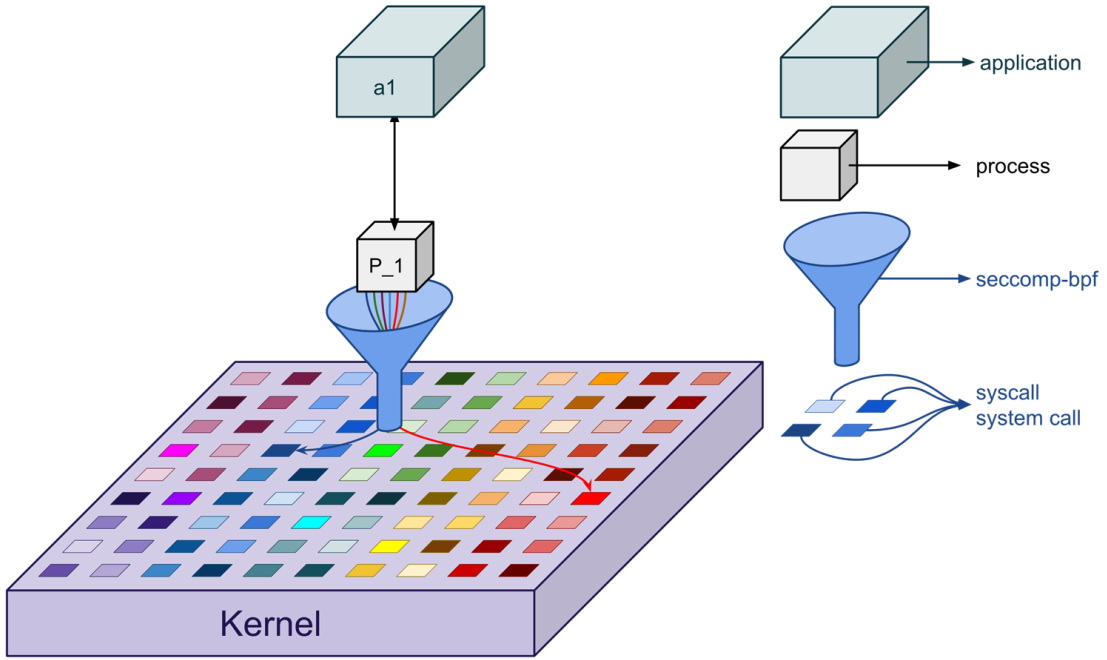
Kernel ezaugarriak – syscalls eta seccomp-bpf

- *syscalls* edo *system call*
  - Edozein aplikaziok kernelarekin komunikatu nahi duenean erabiltzen duen komunikazio interfazea
  - *syscall* kopurua arkitekturaren araberakoa da
    - Egun 300 bat existitzen dira
  - Adibidez
    - read, write, open, execve, chown, clone



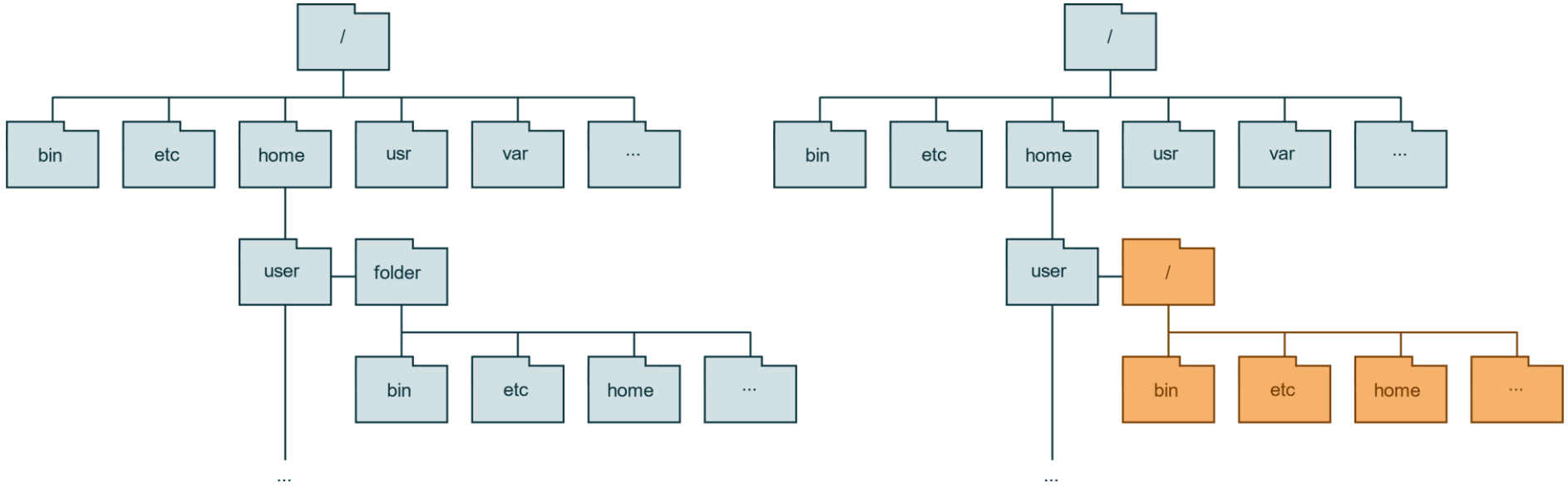
Kernel ezaugarriak – syscalls eta seccomp-bpf

- *seccomp-bpf* edo ***sec**ure **comp**uting **b**erkeley **p**acket **f**ilter*
  - Prozesu batek erabili ditzakeen *syscall* multzoa mugatzeko mekanismoa
    - *syscall* bakoitza exekutatu aurretik pasa behar den filtroa
  - Docker-en defektuzko profil bat existitzen da 40 *syscall* ahalbidetzen dituena



Kernel ezaugarriak – chroot

- *chroot* edo ***ch**ange **root** directory*
  - Prozesu bati erro karpeta aldatzeko *syscall*-a
    - Erro karpeta aldatu ondoren, aldatu berri den karpeta izango da prozesuaren bide-izenen hasiera /
  - Erro karpeta aldatu zaion prozesuaren ondorengo prozesu guztiek heredatzen dute



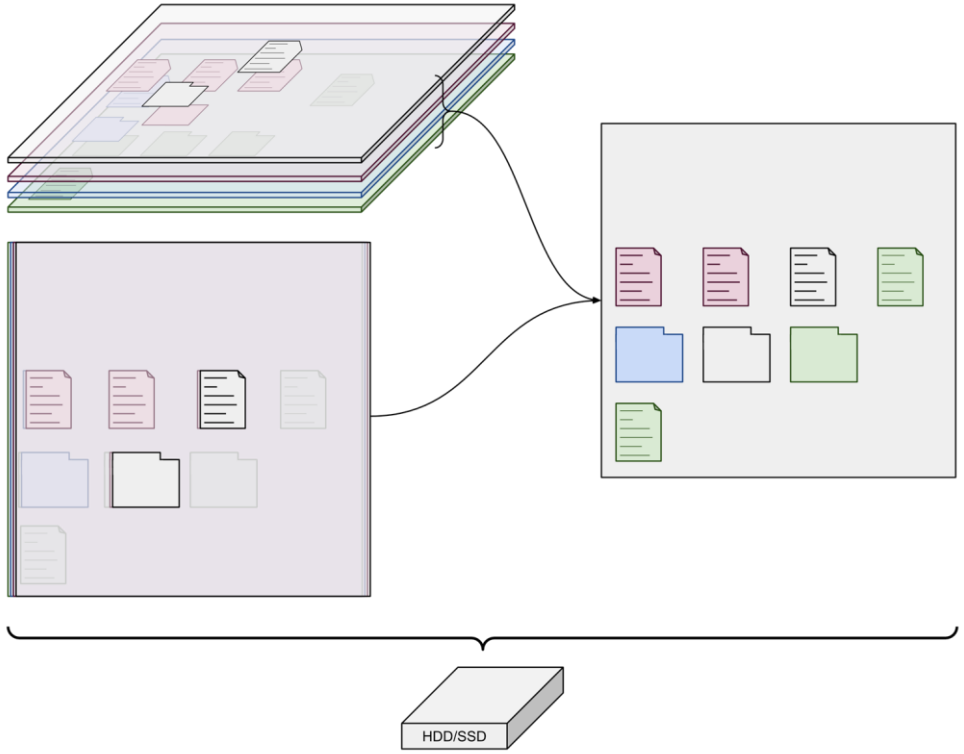
Kernel ezaugarriak – copy-on-write fitxategi sistemak

- COW fitxategi sistemak *snapshot*-ak sortzeko erabiltzen den mekanismoa da
- Normalean, aldatutako datuak bakarrik gordetzen dituzte *snapshot*-ek
- Adibideak
  - btrfs
  - zfs
  - overlayfs

geruzak sortu, cuando ponemos una capa encima de otra capa no podemos cambiar la anterior, y al terminar vemos una combinacion de todas?

Kernel ezaugarriak – ovelaryfs COW fitxategi sistema

- Direktorio zuhaitz bat idatzi eta irakurtzen uzten du irakurri bakarrik egin daitekeen beste direktorio zuhaitz bati gainezarrita egonik
  - Aldaketa denak gorenengo geruzan egiten dira
- Linux kernelaren 3.18 bertsioan gehitu zen



Kernel ezaugarriak – ovelaryfs COW fitxategi sistema

Mondragon Unibertsitatea
Faculty of Engineering

Adibidea 3

Kernel ezaugarriak – ovelaryfs COW fitxategi sistema

- overlayfs fitxategi sistemak nola funtzionatzen duen ikusiko dugu
  - Lehenik eta behin hainbat karpeta sortuko ditugu

```
# mkdir beheko1 beheko2 beheko3 goikoa lanekoa montatutakoa
```

  - Sortutako karpeta batzuetan fitxategi huts batzuk sortuko ditugu

```
# touch beheko1/fit1 beheko2/fit2 beheko3/fit3 goikoa/fit4
```

  - overlayfs fitxategi sistema montatuko dugu

```
# mount -t overlay overlay -o lowerdir=/root/beheko3:/root/beheko2:/root/beheko1,upperdir=/root/goikoa,\
workdir=/root/lanekoa /root/montatutakoa
```

    - Montatu da?

```
# mount
```

Kernel ezaugarriak – ovelaryfs COW fitxategi sistema

Mondragon Unibertsitatea
Faculty of Engineering

- overlayfs fitxategi sistemak nola funtzionatzen duen ikusiko dugu
  - Zer dago montatu berri dugun karpetaren barruan?

```
# ls -l montatutakoa/
```

  - Fitxategi berri bat sortuko dugu

```
# touch montatutakoa/fit5
```

  - Existitzen den fitxategi bati edukia aldatuko diogu

```
# date > montatutakoa/fit3
```

  - Existitzen den fitxategi bat ezabatuko dugu

```
# rm -rf montatutakoa/fit2
```

  - Zein da oraingo egoera?

```
# ls -l montatutakoa/
```

Kernel ezaugarriak – ovelaryfs COW fitxategi sistema

- overlayfs fitxategi sistemak nola funtzionatzen duen ikusiko dugu
  - Karpeta desmontatuko dugu

```
# umount /root/montatutakoa
```

  - Zein aldaketa gertatu dira? Non?

```
# ls -l montatutakoa/
# ls -l beheko1/
# ls -l beheko2/
# ls -l beheko3/
# ls -l goikoa/
```

Kontenedoreak

Mondragon Unibertsitatea
Faculty of Engineering

Ariketa 3

1.2

Docker

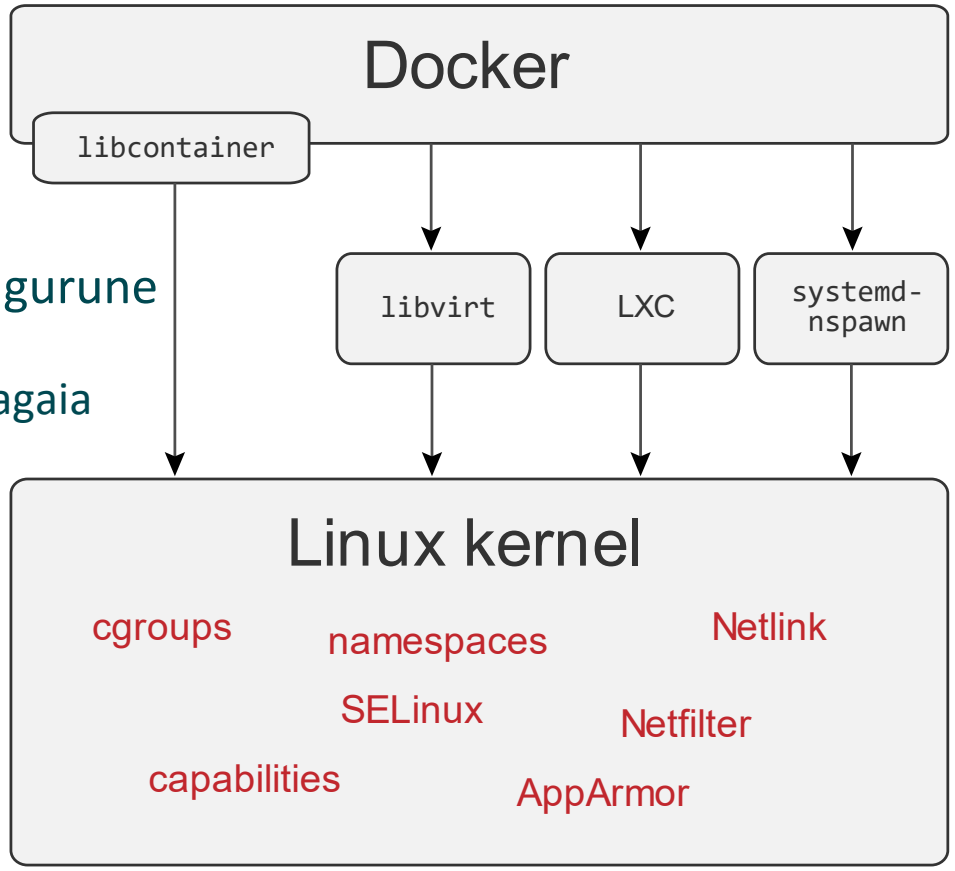
Kontenedoreak

Docker

- Docker
  - *Platform as a Service* (PaaS) produktu sorta, softwarea kontenedore deitzen dugun paketeetan banatzen laguntzen diguna
  - Docker, Inc. enpresak garatua, 2013an kaleratu zen
  - Apache License 2.0 lizentzia dauka

  - Hasiera batean LXC erabiltzen duen exekuzio ingurune bezala
    - 2014an *libcontainer* Go lengoaian idatzitako osagaia kaleratu zuten

  - Linux kernelaren ezaugarri ezberdinetara sarrera izateko, hainbat interfaze ezberdin erabili ditzake



Docker

- Docker
  - Osagaiak
    - Demonioa edo zerbitzua, dockerd izenekoa
      - docker cli, *command-line interface*, erramintarekin lortzen dugu dockerd-rekin hitz egitea

```
# systemctl status docker
```

    - Irudia
      - Irakurri bakarrik egin daitekeen txantiloia, adibidez Dockerfile bidez sortua
      - Sisteman ditugun irudiak ikusi ahal izateko

```
$/# docker images
```

    - Kontenedorea
      - Iruditik sortutako aplikazio exekuzio ingurunea
      - Exekuzioan dauden kontenedoreak ikusteko

```
$/# docker ps
```

      - Exekuzioan egon diren eta ezabatuak izan ez diren kontenedoreak ikusteko

```
$/# docker ps -a
```

Docker

- Docker
  - Osagaiak
    - Zerbitzua
      - Hainbat konputazio gailutan Docker instalatuta eta beraien artean *cluster* bat osatua dugunean, kotenedoreak konputazio gailu ezberdinen artean eskalatu ditzakegu
      - Honi *docker swarm* deitzen zaio
    - Erregistroak (* Kontenedore erregistro pribatu bat nola sortu eta erabili aurrerago ikusiko dugu)
      - Docker irudien errepositoriak
      - Ofiziala Docker Hub den arren beste erregistro publiko batzuk ere existitzen dira eta pribatuak ere sortu daitezke
        - Adibidez GCP-en edo Proxmox-eko sistema kontenedore batean gure erregistro pribatua sortu dezakegu
      - Docker Hub erregistrotik docker irudi bat lortzeko

```
$/# docker pull <irudi_izena:tag>
```

      - Docker Hub erregistrora docker irudi bat bidaltzeko (aurrez erregistratuta egon behar da)

```
$/# docker push <irudi_izena:tag>
```

Docker

- Docker
  - Osagaiak
    - Dockerfile-a
      - Docker irudiak sortzeko erabili dezakegun errezeta edo formula
      - Lengoaia berezi bat du ondorengo instrukzioekin
        - https://docs.docker.com/engine/reference/builder/

-ADD	-LABEL
-ARG	-MAINTANER
-**CMD**	-ONBUILD
-COPY	-**RUN**
-**ENTRYPOINT**	-SHELL
-**ENV**	-STOPSIGNAL
-EXPOSE	-**USER**
-**FROM**	-VOLUME
-HEALTHCHECK	-**WORKDIR**

Gehien erabiliko ditugunak

      - Behin Dockerfile-a idatzita ondorengo komandoaren bidez sortu dezakegu irudia

```
$/# docker build –t <irudi_izena<:tag>> /Dockerfile/kokalekua
```

Docker

- Docker
  - Osagaiak
    - Kontenedorea
      - Docker irudia sortu ondoren, ondorengo komandoaren bidez exekutatu dezakegu

```
$/# docker run <aukerak> irudia <komandoa> <argumentoak>
```

      - Hainbat aukera ezberdin erabili daitezke kontenedorea exekutatzean
        - https://docs.docker.com/engine/reference/commandline/container_run/

Docker

--add-host
--annotation
-a, --attach
--blkio-weight
--blkio-weight-device
--cap-add
--cap-drop
--cgroup-parent
--cgroupns
--cidfile
--cpu-count
--cpu-percent
--cpu-period
--cpu-quota
--cpu-rt-period
--cpu-rt-runtime
-c, --cpu-shares
--cpus
--cpuset-cpus
--cpuset-mems
-d, --detach
--detach-keys
--device
--device-cgroup-rule
--device-red-bps
--device-read-iops
--device-write-bps
--device-write-iops
--disable-content-trus
--dns
--dns-options
--dns-search
--domainname
--entrypoint
-e, --env
--env-file
--expose
--gpus
--group-add
--health-cmd
--health-interval
--health-retries
--health-start-interval
--health-timeout
--help
-h, --hostname
--init
-i, --interactive
--io-maxbandwidth
--io-maxiops
--ip
--ip6
--ipc
--isolation
--kernel-memory
-l, --label
--label-file
--link
--link-local-ip
--log-driver
--log-opt
--mac-address
-m, --memory
--memory-reservation
--memory-swap
--memory-swappiness
--mount
--name
--network
--network-alias
--no-healthcheck
--oom-kill-disable
--oom-score-adj
--pid
--pids-limit
--platform
--privileged
-p, --publish
-P, --publish-all
--pull
-q, --quiet
--read-only
--restart
--rm
--runtime
--security-opt
--shm-size
--sig-proxy
--stop-signal
--stop-timeout
--storage-opt
--sysctl
--tmpfs
-t, --tty
--ulimit
-u, --user
--userns
--uts
-v, --volume
--volume-driver
--volumes-from
-w, --workdir

Docker

- Docker
  - Osagaiak
    - Kontenedorea
      - Exekutatu ondoren, ondorengo komandoaren bidez ikusi ditzakegu exekuzio parametroak

```
$/# docker inspect <aukerak> kontenedorea
```

      - Hainbat aukera ezberdin erabili daitezke kontenedorea exekutatzean
        - https://docs.docker.com/engine/reference/commandline/inspect/

-f, --format
-s, --size
--type

Docker

- Docker
  - Osagaiak
    - Kontenedorea
      - Exekuzioan dagoen kontenedore baten, ondorengo komandoaren bidez exekutatu ditzakegu komandoak

```
$/# docker exec <aukerak> kontenedorea komandoa <argumentoak>
```

      - Hainbat aukera ezberdin erabili daitezke kontenedorea exekutatzean
        - https://docs.docker.com/engine/reference/commandline/inspect/

-d, --detach
--detach-keys
-e, --env
--env-file
-i, --interactive
--privileged
-t, --tty
-u, --user
-w, --workdir

Adibidea 4

Docker

- python3 instalatuta duen eta Debian baten oinarritzen den irudia
  - Dockerfile-a

```
FROM debian:bookworm-20240211-slim

ENV DEBIAN_FRONTEND noninteractive

RUN apt-get update \
 && apt-get install -y \
    python3 \
    --no-install-recommends \
 && rm -rf /var/lib/apt/lists/* /tmp/* /var/tmp/*

ENV USER user

ENV HOME /home/$USER

RUN groupadd -g 1000 $USER \
 && useradd -u 1000 -g 1000 --create-home --home-dir $HOME $USER \
 && chown -R $USER:$USER $HOME

WORKDIR $HOME

USER $USER

ENTRYPOINT ["bash"]
```

Docker

- python3 instalatuta duen eta Debian baten oinarritzen den irudia
  - Dockerfile-tik irudia sortzeko

```
# docker build -t python3 /root/
```

  - Sortutako irudia ikusteko

```
# docker images
```

    - Zein tag du sortutako irudiak?
    - Zein da tamaina?

Docker

- python3 instalatuta duen eta Debian baten oinarritzen den irudia
  - Iruditik kontenedorea exekutatzeko

```
# docker run -it --rm --name python3 python3
```

    - Zein erabiltzailek exekutatzen du kontenedorea?
    - Zein da *hostname*-a?

  - Python3-ko http.server moduloa erabiliz web zerbitzari bat exekutatuko dugu
    - Lehenik eta behin web orri bat sortuko dugu

```
user@<container_id> echo \
'<html><head><title>python3</title></head><body><center>python3</center></body></html>' > index.html
```

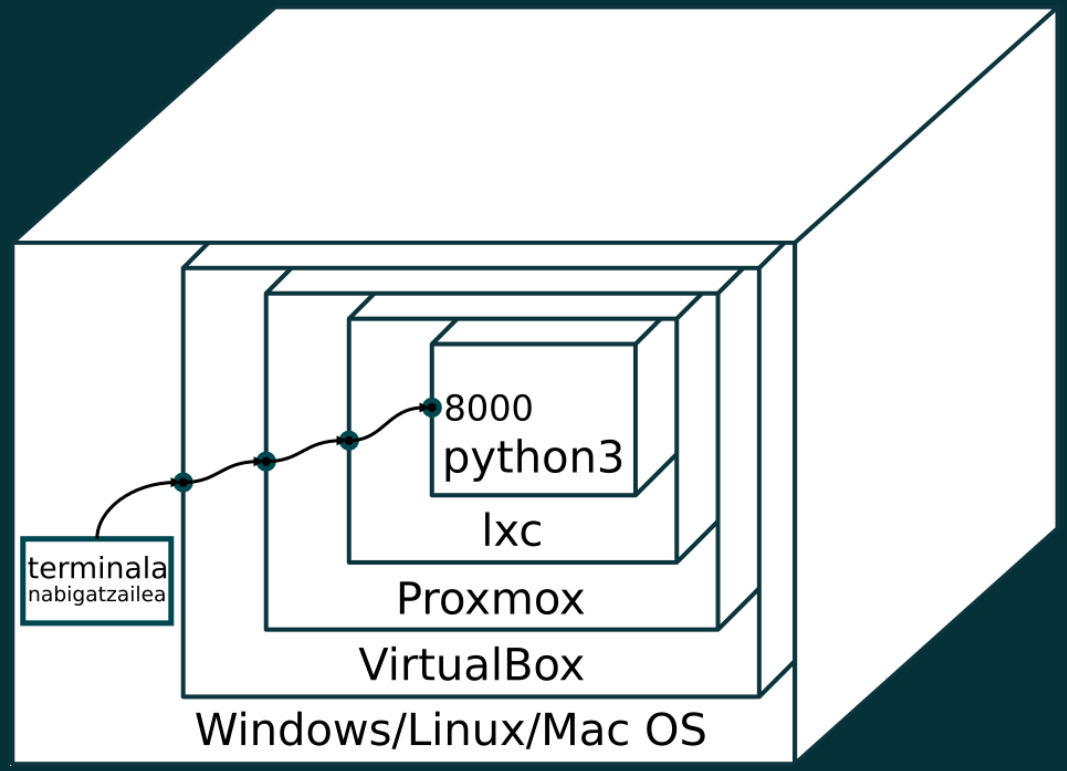
    - Ondoren web zerbitzaria exekutatuko dugu

```
user@<container_id> python3 -m http.server &
```

    - Konektatu al gaitezke gure host makinatik (Windows/Linux/Mac OS) zerbitzari honen 8000 portura?

Docker

- python3 instalatuta duen eta Debian baten oinarritzen den irudia
    - Ondorengo Sistema habiaraketa, *anidación*, daukagu
        - Zer egin beharko genuke?



Docker

- python3 instalatuta duen eta Debian baten oinarritzen den irudia
    - Zer egin beharko genuke?
        1. VirtualBox-en SSH-rentzat portu berbideraketa konfiguratu
        2. Windows-etik Proxmox-eko makinara, pve-ra, SSH tunel bat ireki

```
C:\> ssh -p 2224 -L 0.0.0.0:8000:127.0.0.1:8000 root@127.0.0.1
```

        3. Proxmox-eko makinatik, pve-tik, lxc-ra bigarren SSH tunel bat ireki

```
root@pve:~# ssh -L 8000:<python3_kontenedore_ip-a>:8000 root@<lxc_ip-a>
```

        4. Windows-en nabigatzaile batean localhost:8000 portura konektatu

    - SSH tunel bakarrarekin posible izango al litzateke?
        1. Horretarako kontenedorea exekutatzean 8000 portua publiko egin beharko genuke

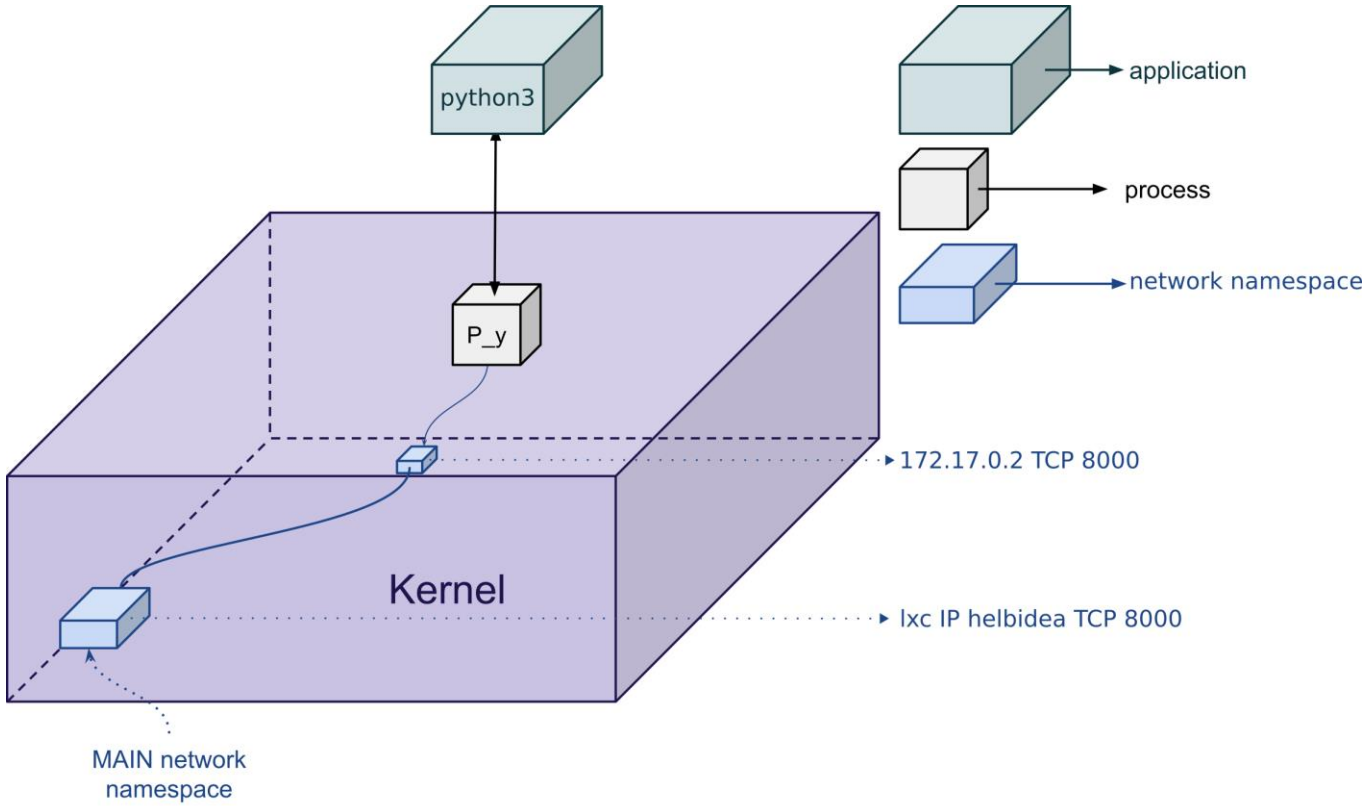
```
# docker run -it --rm --name python3 -p 8000:8000 python3
```

        2. Eta Windows-etik Proxmox-eko makinara, pve-ra, egiten dugun SSH tunela aldatu

```
C:\> ssh -p 2224 -L 0.0.0.0:8000:<lxc_ip-a>:8000 root@127.0.0.1
```

Docker

- python3 instalatuta duen eta Debian baten oinarritzen den irudia
    - Zer egiten du -p 8000:8000 aukerak?



Docker

- python3 instalatuta duen eta Debian baten oinarritzen den irudia
    - SSH tunel bakarrarekin posible izango al litzateke?
        1. Beste aukera bat kontenedorea host-aren sare *namespace*-an exekutatzea da

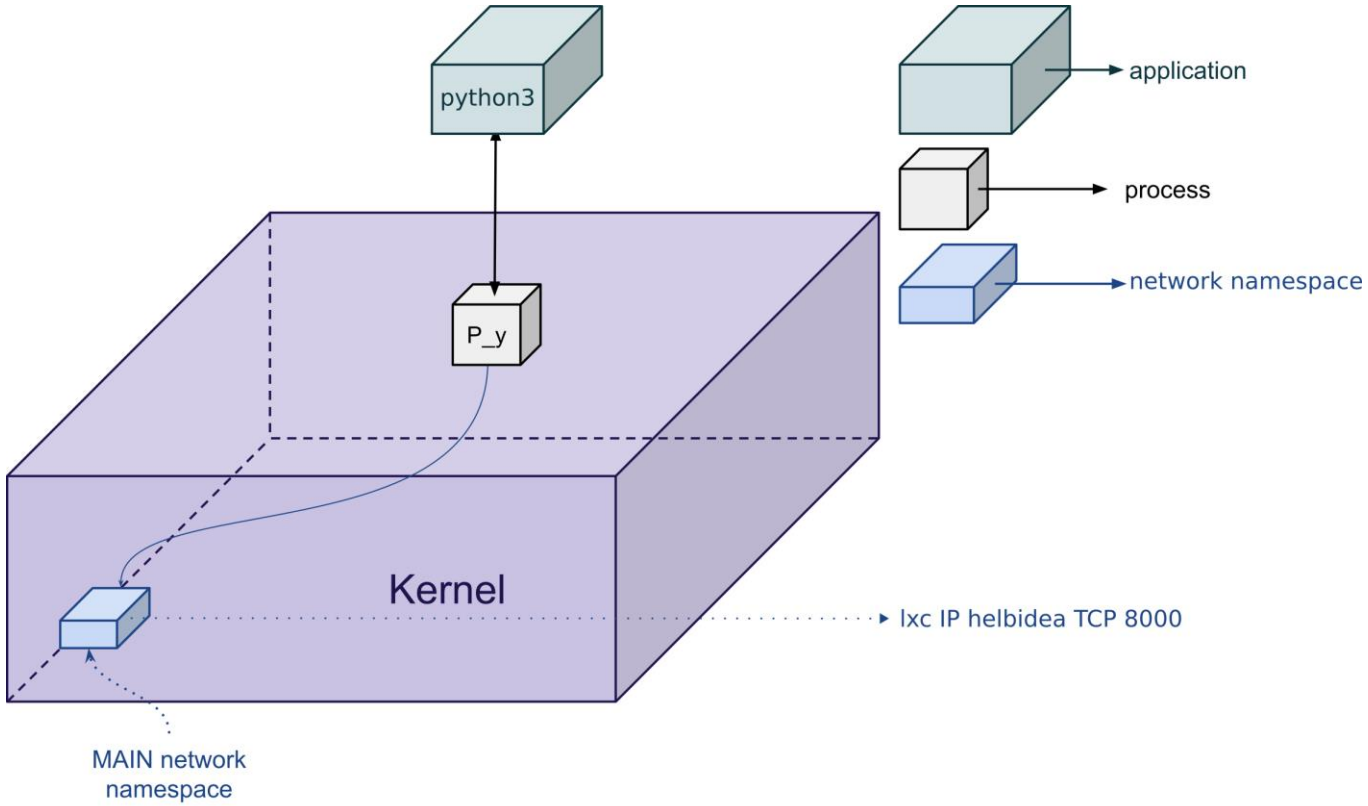
```
# docker run -it --rm --name python3 --network host python3
```

        2. Eta Windows-etik Proxmox-eko makinara, pve-ra, egiten dugun SSH tunela aldatu

```
C:\> ssh -p 2224 -L 0.0.0.0:8000:<lxc_ip-a>:8000 root@127.0.0.1
```

Docker

- python3 instalatuta duen eta Debian baten oinarritzen den irudia
    - Zer egiten du --network host aukerak?



Docker

- python3 instalatuta duen eta Debian baten oinarritzen den irudia
    - Zer gertatzen zaio kontenedorean sortzen dugun web orriari?
        - Kontenedorea itzali eta berriz exekutatzen badugu web orria existitzen al da?

```
user@<container_id>~:$ exit
```

```
# docker run -it --rm --name python3 -p 8000:8000 python3
```

Docker

- python3 instalatuta duen eta Debian baten oinarritzen den irudia
    - Kontenedore barruan egiten ditugun aldaketak gordetzea nahi baditugu, aukerarik “onena” *volumen* bat partekatzea da
        - Nola partekatzen da *volumen* bat?

```
# docker run -it --rm --name python3 -p 8000:8000 -v /root/webgunea:/home/user/ python3
```

        - Kontenedorea exekutatu aurretik *edo ondoren* web orria sortu beharko genuke

```
user@<container_id> echo \
'<html><head><title>python3</title></head><body><center>python3</center></body></html>' > index.html
```

        - Errorerik ematen al digu web orria sortzeak?
            - Zergaitik?

Docker

- python3 instalatuta duen eta Debian baten oinarritzen den irudia
    - Nola esan diezaiokegu kontenedoreari exekutatzen denean zuzenean python3 bidezko web zerbitzaria martxan jarri dezan?
        - Dockerfile-ko *Entrypoint*-a aldatu beharko dugu

```
…

ENV USER user

ENV HOME /home/$USER

RUN groupadd -g 1000 $USER \
 && useradd -u 1000 -g 1000 --create-home --home-dir $HOME $USER \
 && chown -R $USER:$USER $HOME

WORKDIR $HOME

USER $USER

ENTRYPOINT ["/usr/bin/python3", "-m", "http.server"]
```

Docker

- python3 instalatuta duen eta Debian baten oinarritzen den irudia
    - Nola esan diezaiokegu kontenedoreari exekutatzen denean zuzenean python3 bidezko web zerbitzaria martxan jarri dezan?
        - Dockerfile-tik kontenedore irudia sortu

```
# docker build -t python3 /root/
```

        - Eta kontenedorea exekutatu

```
# docker run -it --rm --name python3 -p 8000:8000 -v /root/webgunea:/home/user/ python3
```

    - Zer gertatzen da -it aukera erabili beharrean -d aukera erabiltzen badugu?

```
# docker run -d --rm --name python3 -p 8000:8000 -v /root/webgunea:/home/user/ python3
```

        - Nola ikusi dezakegu zein kontenedore exekutatzen ari den?

```
# docker ps
```

Docker

- python3 instalatuta duen eta Debian baten oinarritzen den irudia
    - Nola ikusi dezakegu zeintzuk diren kontenedorearen exekuzio parametroak?

```
# docker inspect python3
```

    - Nola konektatu gaitezke exekuzioan dagoen kontenedore batetara?

```
# docker exec -it python3 bash
```

    - Nola konektatu gaitezke exekuzioan dagoen kontenedore batetara *root* bezala?

```
# docker exec -u 0 -it python3 bash
```

Ariketa 4

Mondragon Unibertsitatea
Faculty of Engineering

Eskerrik asko
Thank you

Iñaki Garitano
igaritano@mondragon.edu

Data Analysis and Cybersecurity Research Group
Telematics and Cybersecurity Area
Electronics and Computing Department
Mondragon University - Faculty of Engineering
Goiru, 2; 20500 Arrasate - Mondragon (Gipuzkoa), Spain
Tel. : +(34) 647503682 / +(34) 943794700 + Ext. 8119